Calvin Li

calvinliqh@gmail.com
(415)271-1380
Seattle, WA
github.com/calvin-li

University of California, Davis

2014 - B.S. Computer Science
2016 - M.S. Computer Science

Research Project: Random testing of database engines
June - September 2014

- Randomly generate test cases (SQL queries) in order to look for bugs in database engines
- Test for correctness by transforming queries to equivalent ones and comparing
- Can also test correctness by running queries on multiple engines

Programming Languages and Technologies

Professional Experience

Python
C#
Azure
Kubernetes
DataDog
Google Cloud
GitLab
PagerDuty

Proficient

Java
Kotlin
LaTex

Experience

C
Swift
C++
JavaScript
Helm

Personal Projects

Quest
github.com/calvin-li/Quest

- I do Android development as a hobby
- Entirely self-taught
- This is an app I made to help me track tasks using Android's notification system
- Source code on Github, also on the Play Store

Microsoft Corporation

June 2015 - September 2015
Software Engineer Intern, Microsoft Authenticator App

- iOS app for Microsoft's MFA service
- Added usability features (fingerprint authentication and Apple Watch integration)
- Main app coded in Objective-C, watch app coded in Swift

June 2016 - August 2017
Software Engineer, Azure AD Graph

- Rest API for Azure Active Directory
- Added publisher domain feature to Applications API
  - Lets application owners specify their a domain to show users when asking for permissions
  - Responsible for implementation and test design
- Maintained the product's C# library
  - Helps developer make requests to the API without having to craft HTTP Rest requests
  - Inherited sole ownership upon joining team
  - Fixed bugs, cleaned up the code, and added automation
  - Worked with users to transition them to successor product (Microsoft Graph)

August 2017 - June 2019
Software Engineer, Microsoft Graph

- Proxy API for AAD Graph and other Microsoft APIs
- Split time between feature development and maintenance of 4 nines uptime
- Expanded service into new cloud environment (US government)
- Took ownership of our integration testing service
- Part of the on-call rotation
  - Responded to incidents from automated monitoring or users 24/7
  - Used experience to analyze health of service and make improvements to both project setup and livesite setup
  - Designed and implement improvements to our alerting and troubleshooting.

Zume, Inc

November 2019 - March 2020
Software Engineer

- Worked on web app for tracking production at packaging factory

  - Improved CI/CD of system, bringing it in line with company wide standards
  - Learned some web development and containers

Zonar Systems, Inc

March 2020 - Present
Software Engineer II, OEM team

- Maintained and improved existing OEM services
  - Services generally ingested data from partner hardware, processed, then inserted data into Zonar's systems
  - Required collaborating directly with multiple different partner companies (eg DTNA, Cummins), each with their own hardware and systems
  - Learned the systems well enough to be part of the on-call rotation within 3 months of joining
  - Improved DevOps and helped uplift the services to the cloud
- Helped develop Light Duty TCU (Telematics Control Unit) system
  - A simpler, cheaper version of Zonar's existing product geared towards light vehicles (eg cars and pickups) instead of large trucks
  - Collaboration with parent company (Continental Ag), who developed the hardware
  - System developed from the ground up as a part of a larger OEM platform, designed to eventually replace all the various services we currently have
  - Saw the product through from planning to launch (Dec. 2021) and beyond
  - Developed a load testing system that takes previously recorded input and replays it to simulate a system under load
- Helped plan and develop the OEM Platform
  - This is a restructure of our services to be more generalized and use more shared code
  - Beginning with Light Duty TCU, and then integrating out older services
  - The idea is to create a common framework to ingest data from various sources, process it, and then send it to our backend
- OEM systems made up of micro-services, mix of Google Cloud Functions and containers in GKE. Data handled using MemoryStore (Google Redis) and Google DataStore.
  - Python used throughout
  - Heavy use of GitLab CI for continuous pre-prod deployments, and quick prod releases

  - Use Terraform to manage Google Cloud Infrastructure through code
- They're based in Seattle, but have been primarily remote since I joined